SdsFalcon

Version 2.0.1

Sixdof Space

1 Sixdof Falcon Documentation 1

1.1 Introduction . . . 1
1.2 Network Setup . . . 2
1.3 Coordinate Systems . . . 4
1.4 Tracking Data . . . 5
1.5 6DOF Data . . . 5
1.6 Relative Angle Data . . . 6
1.7 Change Log . . . 6
1.8 Trouble Shooting Guide . . . 7
1.8.1 Missing Libraries . . . 7
1.8.2 Ping Failed . . . 7
1.8.3 Firewall . . . 7
1.8.4 Underprocessing . . . 7

2 Class Index 8

2.1 Class List . . . 8

3 File Index 9

3.1 File List . . . 9

4 Class Documentation 10

4.1 Sds::Falcon::Beacon Struct Reference . . . 10
4.1.1 Detailed Description . . . 10
4.2 Sds::Falcon::Config Struct Reference . . . 10
4.2.1 Detailed Description . . . 11
4.3 Sds::Falcon::ConfigGyroCalibration Struct Reference . . . 11
4.3.1 Detailed Description . . . 11
4.4 Sds::Falcon::CurrentShutter Struct Reference . . . 12
4.4.1 Detailed Description . . . 12
4.5 Sds::Falcon::DataPlaybackSettings Struct Reference . . . 12
4.5.1 Detailed Description . . . 12
4.6 Sds::Falcon::DevFeatures Class Reference . . . 13
4.6.1 Detailed Description . . . 13
4.7 Sds::Falcon::DynamicMessage Struct Reference . . . 13
4.7.1 Detailed Description . . . 13
4.8 Sds::Falcon::FalconManager Class Reference . . . 14
4.8.1 Detailed Description . . . 15
4.8.2 Member Function Documentation . . . 15
4.8.2.1 getGyroCalibration() . . . 15
4.8.2.2 initialize() [1/2] . . . 16
4.8.2.3 initialize() [2/2] . . . 16
4.8.2.4 loadDefaultShutterControl() . . . 17
4.8.2.5 loadShutterPlugin() . . . 17

4.8.2.6 setShutterControl() . . . 17
4.8.2.7 swapMap() . . . 17
4.9 Sds::Falcon::FieldOfViewReport Struct Reference . . . 18
4.9.1 Detailed Description . . . 18
4.10 Sds::Falcon::GyroOffset Struct Reference . . . 18
4.10.1 Detailed Description . . . 19
4.11 Sds::Falcon::Heartbeat Struct Reference . . . 19
4.11.1 Detailed Description . . . 19
4.12 Sds::ISetShutterListener Class Reference . . . 19
4.12.1 Detailed Description . . . 20
4.13 Sds::IShutterControl Class Reference . . . 20
4.13.1 Detailed Description . . . 20
4.13.2 Member Function Documentation . . . 20
4.13.2.1 registerSetShutterListener() . . . 20
4.13.2.2 setShutterFixed() . . . 21
4.13.2.3 updateShutter() . . . 21
4.14 Sds::Falcon::NetworkConfig Struct Reference . . . 21
4.14.1 Detailed Description . . . 22
4.15 Sds::OpticalPeak Struct Reference . . . 22
4.15.1 Detailed Description . . . 22
4.16 Sds::Falcon::Pose6Dof Struct Reference . . . 23
4.16.1 Detailed Description . . . 23
4.17 Sds::Falcon::RelativeAngle Struct Reference . . . 24
4.17.1 Detailed Description . . . 24
4.18 Sds::Falcon::StatusMessage Struct Reference . . . 24
4.18.1 Detailed Description . . . 25
4.19 Sds::Falcon::Version Class Reference . . . 25
4.19.1 Detailed Description . . . 25
4.19.2 Member Function Documentation . . . 25
4.19.2.1 getMajor() . . . 25
4.19.2.2 getMinor() . . . 26
4.19.2.3 getPatch() . . . 26
4.19.2.4 getString() . . . 26
4.19.2.5 isAtLeast() . . . 26
4.19.2.6 isEqual() . . . 27

5 File Documentation **28**

5.1 DevFeatures.h File Reference . . . 28
5.1.1 Detailed Description . . . 29
5.1.2 Macro Definition Documentation . . . 29
5.1.2.1 DEV_FEATURE_LIST . . . 29
5.1.3 Enumeration Type Documentation . . . 29

5.1.3.1 ParamType . . . 29
5.1.4 Function Documentation . . . 29
5.1.4.1 createDevFeatures() . . . 29
5.1.4.2 loadDevFeatures() . . . 30
5.2 DevFeatures.h . . . 30
5.3 SdsFalcon.h File Reference . . . 31
5.3.1 Detailed Description . . . 32
5.3.2 Typedef Documentation . . . 33
5.3.2.1 CallbackHandle . . . 33
5.3.2.2 RelativeAngleCollection . . . 33
5.3.3 Enumeration Type Documentation . . . 33
5.3.3.1 CallingLayer . . . 33
5.3.3.2 DataPlaybackMode . . . 33
5.3.3.3 FalconEventCode . . . 34
5.3.3.4 Severity . . . 34
5.3.3.5 TrackingState . . . 34
5.3.4 Function Documentation . . . 35
5.3.4.1 createFalconManager() . . . 35
5.3.4.2 getVersion() . . . 35
5.3.4.3 loadConfig() . . . 35
5.3.4.4 loadMap() . . . 36
5.3.4.5 saveConfig() . . . 36
5.3.4.6 to_string() . . . 36
5.4 SdsFalcon.h . . . 37
5.5 SdsShutterControl.h File Reference . . . 40
5.5.1 Detailed Description . . . 41
5.6 SdsShutterControl.h . . . 41

Chapter 1

Sixdof Falcon Documentation



1.1 Introduction

Sixdof Space has created an optical tracking solution offering tracking with cm level accuracy both indoors and outdoors in direct sunlight. Our patented technology leverages infrared lighting to serve as location beacons. The Falcon SDK contains algorithms that will report the 6DOF positional data of the sensor relative to the beacons, 400 times a second. By connecting the sensor to your drone, robot, or VR headset, the Sixdof Falcon tracking system can provide full 6DOF tracking to your platform.

SdsFalcon is a shared library that enables users to interface with the Sixdof tracking technology. Drone guidance is the main application of this library, however other applications that require a programmatic interface to the Sixdof technology can use this library as well.

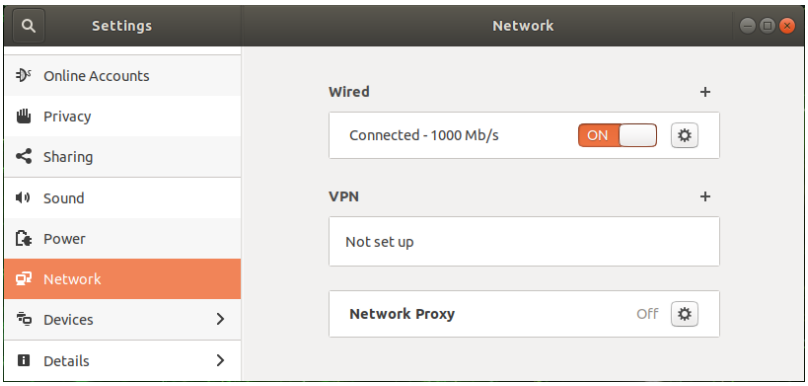
The SdsFalcon shared library is available on the following platforms:

- Windows
- Linux Ubuntu
- Linux Raspberry Pi OS (32 bit)
- Linux Raspberry Pi OS (64 bit)
- Nvidia Jetson

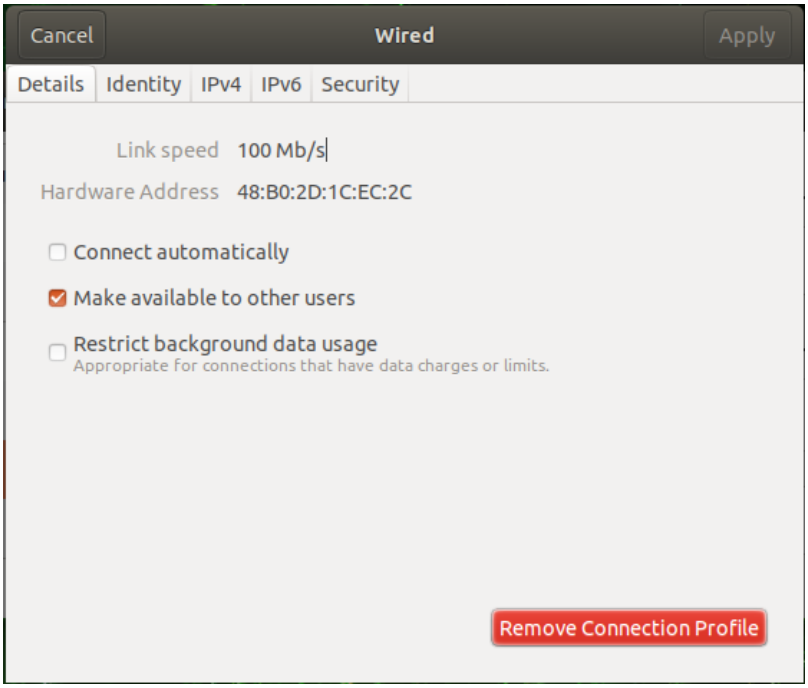
If you want to use the SdsFalcon on another platform, please contact the Sixdof team.

1.2 Network Setup

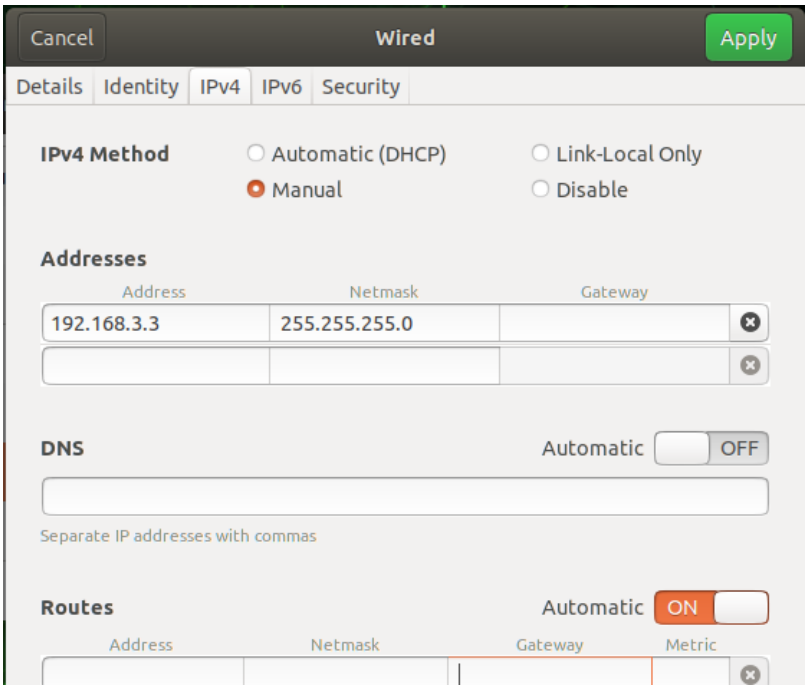
To connect the sensor to the host computer, first connect the sensor's USB to a 5V power source. Next connect the RJ45 port to a network port on the host computer, or via a USB to Ethernet dongle. On Linux based computers, you must set up the network before operation of the sensor. Additionally, to achieve best performance on Linux platforms, Wifi should be turned off during operation. The net-tools package is required to connect to the sensor. You can test if you have the net-tools package by typing ¨ifconfig¨ into the command line, if the response says ¨command not found¨ please install the net-tools package via the command: ¨sudo apt install net-tools¨. The following instructions are for Ubuntu based distributions, the Raspberry Pi 3b instructions are shown below. First navigate to the Network Settings and ensure the ¨Wired¨ connection is set to ¨ON¨:



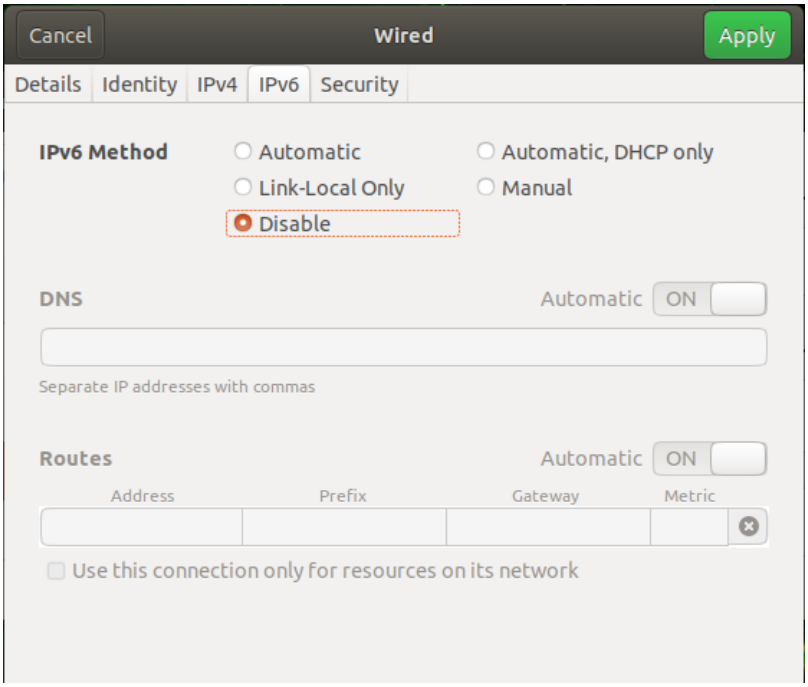
If you are using a USB dongle there will be another row labeled ¨USB Ethernet¨, set that to ¨ON¨. Next click the gear button to go to the advanced settings:



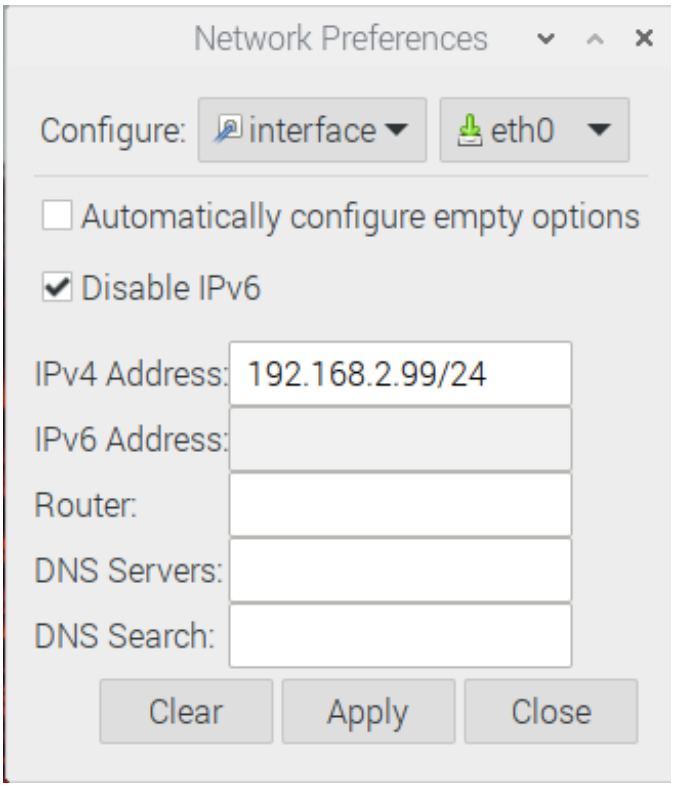
In the ¨Details¨ tab, deselect the checkbox that says ¨Connect automatically¨. Next navigate to the IPv4 tab and set the ¨IPv4 Method¨ to ¨Manual¨ and set the ¨Address¨ and ¨Netmask¨. The Address can be any IP address that you choose, however it should not clash with any other networks on the host computer. To avoid IP address clashes as much as possible you can set the Netmask to 255.255.255.0. Additionally, set the ¨DNS Automatic¨ to ¨OFF¨ as shown below:



Finally, navigate to the IPv6 tab and for the ¨IPv6 Method¨ select ¨Disable¨:



On Raspberry Pi 3b, navigate to the ¨Network Preferences¨ menu, next to ¨Configure¨ leave the first drop down on the ¨interface¨ option, and on the second drop down select the network that you are connected to, typically ¨eth0¨. If you are using a USB dongle the network may have a different name. Deselect the options that says ¨Automatically configure empty options¨, and select ¨Disable IPv6¨. Next fill in an IP Address that you selected and put ¨/24¨ at the end to indicate that you want a submask of 255.255.255.0. An example is shown below:



1.3 Coordinate Systems

Before using the Sixdof tracking system it is important to understand the coordinate systems involved. The sensor coordinate system is defined according to the image below:

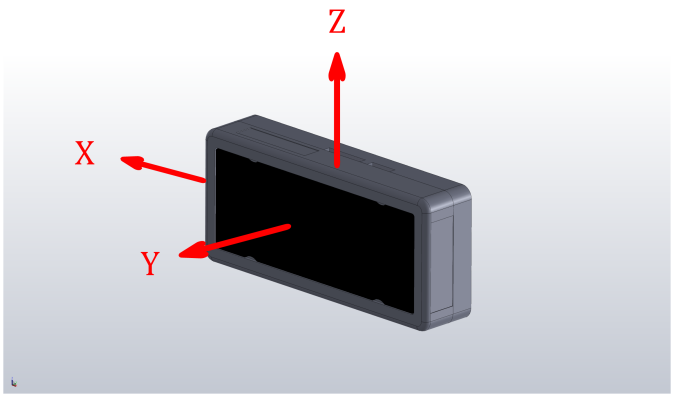


Figure 1.1 Sensor coordinate system

Where the positive x-axis points to the right of the sensor, the positive y-axis point out from the sensor and the positive z-axis completes the right handed system. The field of view can be approximated by a +/- 60 degree vision cone around the y-axis.

In many applications, the user is interested in the location of the host platform (drone, or robot), and not just the location of the sensor. In these applications, it is the user's responsibility to do the rigid body transformation from the sensor to the host platform, based on how the sensor is mounted on the platform.

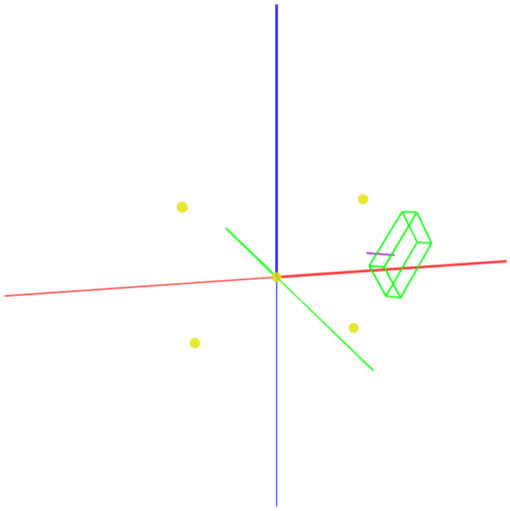
1.4 Tracking Data

There are two representations of tracking data available simultaneously:

- 6DOF: the full six degrees of freedom (position and orientation) relative to the map.
- Relative Angle: the angles of each beacon relative to the sensor, this is intended for applications where the sensor is far from the beacons.

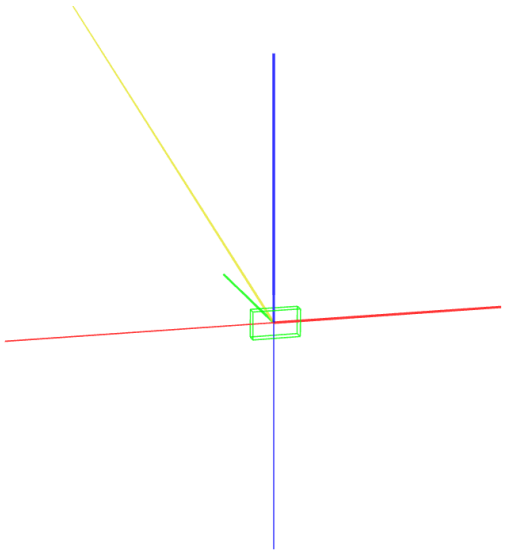
In addition to tracking, a method is provided to enable the user to do gyroscope calibration. During gyroscope calibration, the sensor must be still. In many applications, it is desirable to initialize the system while already in motion. Therefore, gyroscope calibration can be done in advance to the mission and the calibrated values can be fed in at run time. Alternatively, the gyro calibration step can be ignored, this is sufficient in most cases.

1.5 6DOF Data



The 6DOF data presents the position of the sensor tracked relative to the map. The map is made up of multiple IR beacons that flash unique ids, multiple beacons are driven by a single beacon driver board. The user must place these beacons in a rigid formation and provide the X, Y, Z location of each beacon. The unique id of each beacon is provided on each LED individually. The user can define the coordinate system of the map in any way they choose, however it must be a right handed coordinate system. When designing a map, it is important to ensure the lights are not co-linear. Keep in mind the desired area of operation, try to design the map so that there will always be at least 3 beacons in the field of view of the sensor.

1.6 Relative Angle Data



The Relative Angle data presents the angle of a beacon tracked relative to the coordinate system of the sensor. For example, this data can be used to guide a drone from a high elevation all the way down to a landing pad. This data does not provide any orientation information of the senor. Additionally, due to the far distance of the beacon, the baseline on the sensor is insufficient to calculate the distance directly. The intensity of the beacon and the peak width in pixels are reported in this data in order to get a rough estimate of the distance of the beacon.

1.7 Change Log

Version	Date	Notes
2.0.1	08/03/2026	Bug fix to Relative Angles mode.
2.0.0	08/01/2026	6DOF and Relative Angles run simultaneously. Removed Relative Beacon. Added Playback. Added DevFeatures. Removed ShutterSettings from the config.
1.7.4	19/10/2025	Algorithm changes including: kipa-pro support, localization uses P3P algorithm, changed gyro noise parameters, better velocity estimation.
1.7.3	N/A	Fix for carnea member in ShutterControl, added getShutterSettingsDefault so that getShutterSettingsAuto can refer to pure auto shutter mode.
1.7.2	18/06/2025	Fixes for static ip connection.
1.7.1	12/05/2025	Better peak height estimation.
1.7.0	24/11/2024	Added Dynamic Messages, changed beacon Ids to uint32_t, and added support for shutter plugins
1.6.0	21/07/2024	Re-branded to Sixdof Falcon
1.5.5	28/05/2024	Added more elaborate check for the right firmware version, better network debugging tools, bug fix for Raspberry Pi platforms.
1.5.3	29/02/2024	Added NetworkAdvancedConfig, getFirmwareVersion, skip_gyro_calibration is now defaulted to true
1.5.2	20/02/2024	Added expected_beacon_ids to ConfigRelativeBeacon and ConfigRelativeAngle, also changed shutter backed, deprecated setShutter function
1.5.1	01/02/2024	Added skip_gyro_calibration flag to Config6Dof

Version	Date	Notes
1.5.0	20/12/2023	Added setShutter feature, current shutter callback, put shutter_settings into each of the config objects
1.4.1	27/11/2023	Added swapMap feature
1.4.0	05/11/2023	Added Relative Angle feature
1.3.0	17/10/2023	Added health monitoring feature in the form of Heartbeat callbacks
1.2.1	06/09/2023	Exposed ¨No Gyro¨ mode as a parameter in Config6Dof. Made changes to support multiple sensors each with its own FalconManager
1.2.0	22/08/2023	Changed API so that PoseRelativeBeacons that happen at the same time are bundled together
1.1.2	14/08/2023	Updated RelativeBeacon logic, parameter rel_lights_cooldown_ms is no longer needed, instead we have the parameter field_of_view_cutoff_deg
1.1.1	14/08/2023	Statically link GCC libraries, add MSVC build to the release
1.1.0-beta	24/07/2023	Changed header, included gyro calibration, field of view report features and data logging for replay
1.0.0-beta	01/05/2023	inital beta version

1.8 Trouble Shooting Guide

1.8.1 Missing Libraries

Make sure to always copy the shared library (SdsFalcon.dll on Windows and SdsFalcon.so on Linux) that are provided in the release folder for your platform into the the directory with your program. Alternatively, you can statically link the shared library to your program. Also on Windows, you have the option of adding the shared library to your system PATH.

1.8.2 Ping Failed

Errors relating to ping failing mean that the host computer was unable to connect to the Sixdof sensor. Try the following steps:

1. Ensure the Sixdof sensor is connected to power and the RJ45 ethernet cable is connected to your computer.
2. Look at the red indicator lights that are exposed on the top of the senor casing, the lights should be flashing. If the lights are constant, please contact the Sixdof team.
3. Ensure your NetworkConfig parameters are correct, this is the sensor board number and the network name that the sensor is connected to.
4. Your computer's ARP table is used to connect to the sensor, try clearing the ARP table and trying again. The ARP table can be cleared via a command console with administrator privileges, use the command ¨arp -d¨.

1.8.3 Firewall

If SdsFalcon is reporting errors related to communication issues, it is recommended to turn off your firewall.

1.8.4 Underprocessing

If you are getting the ¨Warning¨ status message that says ¨Not receiving enough buffers from board¨ that means that your program is not getting enough CPU time. On Windows systems this is usually due to a spontaneous virus scan, you can temporarily turn off virus scans from in your ¨Windows Security¨ settings.

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Sds::Falcon::Beacon
A structure representing a beacon with 3D coordinates and an id . . . 10
Sds::Falcon::Config
A structure representing configuration settings for the tracking . . . 10
Sds::Falcon::ConfigGyroCalibration
A structure representing configuration settings for gyro calibration . . . 11
Sds::Falcon::CurrentShutter
A structure representing the current shutter value . . . 12
Sds::Falcon::DataPlaybackSettings
The configuration settings of the data playback feature . . . 12
Sds::Falcon::DevFeatures
Class for managing runtime parameters of the Sixdof Falcon tracking system . . . 13
Sds::Falcon::DynamicMessage
A structure representing a dynamic message . . . 13
Sds::Falcon::FalconManager
Class for managing the Sixdof Falcon tracking system . . . 14
Sds::Falcon::FieldOfViewReport
A report of the beacons in the field of view . . . 18
Sds::Falcon::GyroOffset
A structure representing a single gyro calibration offset for a specific sensor . . . 18
Sds::Falcon::Heartbeat
A structure representing health data of the SDK . . . 19
Sds::ISetShutterListener
Listener interface that allows IShutterControl to pass forward requests to change the shutter . . . 19
Sds::IShutterControl
Shutter control base class . . . 20
Sds::Falcon::NetworkConfig
A structure representing network configuration for the Sixdof sensor . . . 21
Sds::OpticalPeak
A structure representing a summary of the optical data for a single beacon . . . 22
Sds::Falcon::Pose6Dof
A structure representing a 6DOF pose . . . 23
Sds::Falcon::RelativeAngle
A structure representing the relative angle of a beacon . . . 24
Sds::Falcon::StatusMessage
A structure representing a status message . . . 24
Sds::Falcon::Version
A class representing the version of either the shared library, or the sensor firmware . . . 25

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

DevFeatures.h
Header file containing declarations for advanced Dev Features of the Sixdof Falcon library . . . 28
SdsFalcon.h
Header file containing declarations for the Sixdof Falcon library 31
SdsShutterControl.h
Header file containing declarations relating to the Sixdof sensors shutter control 40

Chapter 4

Class Documentation

4.1 Sds::Falcon::Beacon Struct Reference

A structure representing a beacon with 3D coordinates and an id.

`#include <SdsFalcon.h>`

Public Attributes

- double **x**

  x-coordinate in meters.
- double **y**

  y-coordinate in meters.
- double **z**

  z-coordinate in meters.
- uint32_t **id**

  unique id.

4.1.1 Detailed Description

A structure representing a beacon with 3D coordinates and an id.

This structure contains the 3D coordinates (x, y, z) of a beacon in meters and its unique id.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.2 Sds::Falcon::Config Struct Reference

A structure representing configuration settings for the tracking.

`#include <SdsFalcon.h>`

Public Attributes

- NetworkConfig **networkConfig**

  network configuratin for connecting to the sensor. (Required)
- std::vector< Beacon > **map**

  vector of beacons for Sixdof tracking. (Required for receiving 6DOF data)
- std::vector< GyroOffset > **gyroCalibration**

  vector of previously recorded gyro calibration data. (Optional)
- bool **skip_gyro_calibration** { true }

  set to true to skip gyro calibration, this will speed up the initialization process.
- bool **no_gyro_mode** { false }

  flag to disable gyro sensor, this mode should only be used in select cases, please contact the Sixdof team for guidance. (Optional)

4.2.1 Detailed Description

A structure representing configuration settings for the tracking.

This structure contains configuration settings for the tracking. The only required data is the networkConfig, other fields can be left to defaults. To receive 6DOF data, the map member must also be populated.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.3 Sds::Falcon::ConfigGyroCalibration Struct Reference

A structure representing configuration settings for gyro calibration.

`#include <SdsFalcon.h>`

Public Attributes

- int **duration_seconds** { 30 }

  duration of gyro calibration capture in seconds.

4.3.1 Detailed Description

A structure representing configuration settings for gyro calibration.

This structure contains configuration settings for gyro calibration, all members can be left to defaults.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.4 Sds::Falcon::CurrentShutter Struct Reference

A structure representing the current shutter value.

`#include <SdsFalcon.h>`

Public Attributes

- uint8_t **shutter_value**
  current shutter value

4.4.1 Detailed Description

A structure representing the current shutter value.

This structure is output via a callback, and indicates the current shutter value. Shutter values are between 0 and 16, where 0 is the shortest shutter and 16 is the longest.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.5 Sds::Falcon::DataPlaybackSettings Struct Reference

The configuration settings of the data playback feature.

`#include <SdsFalcon.h>`

Public Attributes

- DataPlaybackMode **mode** { DataPlaybackMode::NONE }
  playback mode.
- std::string **path** { "" }
  full path to file to log data to, default value of empty string will not log data. (Optional)

4.5.1 Detailed Description

The configuration settings of the data playback feature.

When mode is NONE then the data playback feature is disabled. When mode is RECORD then data is saved to the path provided. When mode is PLAYBACK then data is read from the path provided and fed to the algorithm.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.6 Sds::Falcon::DevFeatures Class Reference

Class for managing runtime parameters of the Sixdof Falcon tracking system.

`#include <DevFeatures.h>`

Public Member Functions

- virtual void **setParam** (ParamType, double)=0
  Sets a parameter.
- virtual double **getParam** (ParamType)=0
  Gets the current value of a parameter.

4.6.1 Detailed Description

Class for managing runtime parameters of the Sixdof Falcon tracking system.

This class provides an interface for managing the runtime parameters of the Sixdof Falcon tracking system.

The documentation for this class was generated from the following file:

- DevFeatures.h

4.7 Sds::Falcon::DynamicMessage Struct Reference

A structure representing a dynamic message.

`#include <SdsFalcon.h>`

Public Attributes

- bool **valid** { false }
- uint32_t **beacon_id**
  unique id of the source beacon.
- std::string **message**
  message that was transmitted optically.

4.7.1 Detailed Description

A structure representing a dynamic message.

This structure contains a ¨Dynamic¨ message. Dynamic messages are messages that are transmitted to the Sixdof sensor optically, via one of the Sixdof smart driver boards.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.8 Sds::Falcon::FalconManager Class Reference

Class for managing the Sixdof Falcon tracking system.

`#include <SdsFalcon.h>`

Public Member Functions

- virtual bool initialize (const Config &)=0

  Initialize the Sixdof Falcon tracking system.
- virtual bool initialize (const Config &, const DataPlaybackSettings &)=0

  Initialize the Sixdof Falcon tracking system with data playback settings.
- virtual void **start** ()=0

  Starts tracking.
- virtual void **stop** ()=0

  Stops tracking, temporarily.
- virtual Version **getFirmwareVersion** ()=0

  Gets the firmware version of the sensor.
- virtual std::vector< GyroOffset > getGyroCalibration (const ConfigGyroCalibration &)=0

  Perform gyro calibration and retrieve the calibration results.
- virtual Pose6Dof **getPose6Dof** ()=0

  get most recent Pose6Dof.
- virtual RelativeAngleCollection **getRelativeAngleCollection** ()=0

  get most recent RelativeAngle.
- virtual StatusMessage **getStatusMessage** ()=0

  get first StatusMessage that was called but not received yet.
- virtual DynamicMessage **getDynamicMessage** ()=0

  get most recent dynamic message.
- virtual FieldOfViewReport **getFieldOfViewReport** ()=0

  get most recent field of view report.
- virtual CurrentShutter **getCurrentShutter** ()=0

  get most recent shutter value.
- virtual CallbackHandle **registerPose6DofCallback** (const std::function< void(Pose6Dof)> &)=0

  register a function callback that takes Pose6Dof as an input.
- virtual CallbackHandle **registerRelativeAngleCallback** (const std::function< void(RelativeAngleCollection)> &)=0

  register a function callback that takes RelativeAngleCollection as an input.
- virtual CallbackHandle **registerMessageCallback** (const std::function< void(StatusMessage)> &)=0

  register a function callback that takes StatusMessage as an input.
- virtual CallbackHandle **registerDynamicMessageCallback** (const std::function< void(DynamicMessage)> &)=0

  register a function callback that takes DynamicMessage as an input.
- virtual CallbackHandle **registerFieldOfViewReportCallback** (const std::function< void(FieldOfViewReport)> &)=0

  register a function callback that takes FieldOfViewReport as an input.
- virtual CallbackHandle **registerHeartbeatCallback** (const std::function< void(Heartbeat)> &)=0

  register a function callback that takes Heartbeat as an input.
- virtual CallbackHandle **registerCurrentShutterCallback** (const std::function< void(CurrentShutter)> &)=0

  register a function callback that takes a CurrentShutter as an input.
- virtual void **removePose6DofCallback** (const CallbackHandle &)=0

< *remove a Pose6Dof function callback.*

- virtual void **removeRelativeAngleCallback** (const CallbackHandle &)=0

  < *remove a RelativeAngle function callback.*
- virtual void **removeMessageCallback** (const CallbackHandle &)=0

  < *remove a StatusMessage function callback.*
- virtual void **removeDynamicMessageCallback** (const CallbackHandle &)=0

  remove a DynamicMessage function callback.
- virtual void **removeFieldOfViewCallback** (const CallbackHandle &)=0

  < *remove a FieldOfViewReport function callback.*
- virtual void **removeHeartbeatCallback** (const CallbackHandle &)=0

  < *remove a Heartbeat function callback.*
- virtual void **removeCurrentShutterCallback** (const CallbackHandle &)=0

  < *remove a Current Shutter function callback.*
- virtual void swapMap (const std::vector< Beacon > &map)=0

  Dynamically swap the map.
- virtual void setShutterControl (std::shared_ptr< IShutterControl > shutterControl)=0

  Set custom shutter control logic.
- void loadShutterPlugin (const std::string &path)

  Set custom shutter control logic.
- void loadDefaultShutterControl ()

  Set the shutter control logic to the default.

4.8.1 Detailed Description

Class for managing the Sixdof Falcon tracking system.

This class provides an interface for managing Sixdof Falcon tracking. The correct order of function calls is first initialize, then start.

There are two ways to get tracking data and status messages out of the FalconManager:
1) Get functions. these will return the most recent data point. However the getStatusMessage function will return all messages in the order they appeared.
2) Function callbacks. The callbacks provided will get called each time a new data point is provided.

It is recommended to use function callbacks for status messages to ensure all messages are received immediately. It is also recommended to register for status messages before initializing the network, this way we can get all messages from the initialization stage. Each function callback is run on its own thread and it only gets updated once it has finished processing its current data point. This means that long running function callbacks may miss data points that got overshadowed by new data, this is by design.

The FalconManager also allows for the client to do gyro calibration before starting, see the getGyroCalibration() function.

4.8.2 Member Function Documentation

4.8.2.1 getGyroCalibration()

```
virtual std::vector< GyroOffset > Sds::Falcon::FalconManager::getGyroCalibration (
            const ConfigGyroCalibration &  )  [pure virtual]
```

Perform gyro calibration and retrieve the calibration results.

Warning

This function must be called only after the sensor has been initialized, as it requires an active network connection to the device.

This function performs a gyro calibration procedure on the sensor. During calibration, the sensor must remain completely stationary.

The resulting calibration data is stored internally and is also returned to the caller. There are two supported usage patterns:

1. **Reuse calibration in a later initialization (recommended for testing):** Save the returned calibration data and provide it to the `gyro_calibration` member of `Config` during a future initialization. This allows calibration to be performed in advance, eliminating the need for the sensor to be stationary during test startup.
2. **Immediate use without saving the output:** The returned calibration data may be ignored. Since the calibration is stored internally, tracking can be started immediately after this call. In this case, the sensor must be stationary at startup, immediately following initialization.

Parameters

ConfigGyroCalibration	Configuration parameters for the gyro calibration procedure.

Returns

A vector of `GyroOffset` objects containing the gyro calibration data.

4.8.2.2 initialize() [1/2]

```
virtual bool Sds::Falcon::FalconManager::initialize (
            const Config &  )  [pure virtual]
```

Initialize the Sixdof Falcon tracking system.

Initializes the tracking system using the provided configuration. Data playback is disabled.

Parameters

Config	The configuration.

Returns

True if initialization was successful, false otherwise.

4.8.2.3 initialize() [2/2]

```
virtual bool Sds::Falcon::FalconManager::initialize (
            const Config & ,
            const DataPlaybackSettings &  )  [pure virtual]
```

Initialize the Sixdof Falcon tracking system with data playback settings.

Initializes the tracking system using the provided configuration and data playback settings. This overload allows enabling recording or playback of data during initialization.

Parameters

Config	The configuration.
DataPlaybackSettings	The data playback settings (none, record, or playback).

Returns

True if initialization was successful, false otherwise.

4.8.2.4 loadDefaultShutterControl()

```
void Sds::Falcon::FalconManager::loadDefaultShutterControl ( )
```

Set the shutter control logic to the default.

This function can be called dynamically on runtime. If the default shutter logic has been overriden, use this function to return to the default.

4.8.2.5 loadShutterPlugin()

```
void Sds::Falcon::FalconManager::loadShutterPlugin (
            const std::string & path )
```

Set custom shutter control logic.

Set custom shutter control logic, from a compiled plugin. This function can be called dynamically at runtime. It should be called after the tracking is initialized.

A valid plugin is in the form of a shared library (.dll on Windows, and .so on Linux), where the following function is implemented: extern ¨C¨ IShutterControl∗ createShutterControl();

4.8.2.6 setShutterControl()

```
virtual void Sds::Falcon::FalconManager::setShutterControl (
            std::shared_ptr< IShutterControl > shutterControl )  [pure virtual]
```

Set custom shutter control logic.

Set custom shutter control logic, from source code. This function can be called dynamically at runtime.

4.8.2.7 swapMap()

```
virtual void Sds::Falcon::FalconManager::swapMap (
            const std::vector< Beacon > & map )  [pure virtual]
```

Dynamically swap the map.

This enables the user to swap the map during execution. It can only be called after initialize has already been called.

Parameters

map	The new beacon positions.

The documentation for this class was generated from the following file:

- SdsFalcon.h

4.9 Sds::Falcon::FieldOfViewReport Struct Reference

A report of the beacons in the field of view.

`#include <SdsFalcon.h>`

Public Attributes

- std::map< uint32_t, int > **seenBeacons**

  Map of beacon ids to the number of optical sensors that the beacon was seen on.

4.9.1 Detailed Description

A report of the beacons in the field of view.

This structure contains information about which beacons are in the field of view of the sensor, and if they are seen on 1, 2 or 3 optical sensors.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.10 Sds::Falcon::GyroOffset Struct Reference

A structure representing a single gyro calibration offset for a specific sensor.

`#include <SdsFalcon.h>`

Public Attributes

- uint16_t **sensorId** { 0 }

  sensor id.
- std::string **datetime** { "" }

  date time in Y-m-d H:M:S format.
- int8_t **temperature** { 0 }

  temperature in degrees. This is the internal temperature of board, not the temperature of the external environment.
- int16_t **gyro_x** { 0 }

  bias in the x-coordinate of the IMU. Units are in gyro units.
- int16_t **gyro_y** { 0 }

  bias in the y-coordinate of the IMU. Units are in gyro units.
- int16_t **gyro_z** { 0 }

  bias in the z-coordinate of the IMU. Units are in gyro units.
- uint16_t **standard_dev** { 0 }

  standard deviation of IMU noise. Units are in gyro units.

4.10.1 Detailed Description

A structure representing a single gyro calibration offset for a specific sensor.

This structure contains information about gyro calibration offset for a specific sensor.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.11 Sds::Falcon::Heartbeat Struct Reference

A structure representing health data of the SDK.

`#include <SdsFalcon.h>`

Public Attributes

- bool **sensor_communication_ok** { false }
  status of communication with the sensor.
- TrackingState **current_tracking_state** { TrackingState::Initializing }
  state of the tracking.

4.11.1 Detailed Description

A structure representing health data of the SDK.

This structure is output via a callback every second. This enables the user to quickly identify that the SDK is still running. Additionally, the Heartbeat structure contains information regarding the state of communications with the sensor and the tracking algorithm.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.12 Sds::ISetShutterListener Class Reference

Listener interface that allows IShutterControl to pass forward requests to change the shutter.

`#include <SdsShutterControl.h>`

Public Member Functions

- virtual void **setShutterAuto** (uint8_t min_shutter, uint8_t max_shutter)=0
- virtual void **setShutterFixed** (uint8_t shutter)=0

4.12.1 Detailed Description

Listener interface that allows IShutterControl to pass forward requests to change the shutter.

This class is used to listen to shutter change requests to SdsFalcon and SdsCheetah. For all intensive purposes this class can be ignored.

The documentation for this class was generated from the following file:

- SdsShutterControl.h

4.13 Sds::IShutterControl Class Reference

Shutter control base class.

`#include <SdsShutterControl.h>`

Public Member Functions

- virtual void updateShutter (uint8_t currentShutter, const std::vector< OpticalPeak > &peaks)=0

  Custom logic to update the shutter.
- void **setShutterAuto** (uint8_t min_shutter, uint8_t max_shutter)

  Set the shutter to Auto-shutter. Use this function to set the shutter to auto shutter, min and max shutter values must be in the range [0, 16].
- void setShutterFixed (uint8_t shutter)

  Set the shutter to a fixed value.
- void registerSetShutterListener (ISetShutterListener *l)

  Register a shutter listener.

4.13.1 Detailed Description

Shutter control base class.

Inherit from this base class to create custom shutter control logic.

4.13.2 Member Function Documentation

4.13.2.1 registerSetShutterListener()

```
void Sds::IShutterControl::registerSetShutterListener (
            ISetShutterListener * l )  [inline]
```

Register a shutter listener.

This function can be ignored by the client.

4.13.2.2 setShutterFixed()

```
void Sds::IShutterControl::setShutterFixed (
            uint8_t shutter )  [inline]
```

Set the shutter to a fixed value.

Use this function to set the shutter to a fixed shutter value, shutter value must be in the range [0, 16].

4.13.2.3 updateShutter()

```
virtual void Sds::IShutterControl::updateShutter (
            uint8_t currentShutter,
            const std::vector< OpticalPeak > & peaks )  [pure virtual]
```

Custom logic to update the shutter.

override this function with your custom logic, call setShutterAuto or setShutterFixed to change the shutter only call setShutterAuto or setShutterFixed when you need the shutter control to change avoid making shutter changes within a small period of time, otherwise some of the requests may be ignored by the sensor

The documentation for this class was generated from the following file:

- SdsShutterControl.h

4.14 Sds::Falcon::NetworkConfig Struct Reference

A structure representing network configuration for the Sixdof sensor.

```
#include <SdsFalcon.h>
```

Public Attributes

- uint16_t **sensor_id**

  sensor id.
- std::string **sensor_network_name** { "" }

  name of the network the sensor is connected to.
- std::string **sensor_ip** { "" }

  ip address of the sensor (optional). Note, this is for advanced configuration only, and should generally be left to the default value.
- uint16_t **udp_read_port** { 0 }

  udp port to read from the sensor, to be auto-assigned a port leave as 0. (Optional)

4.14.1 Detailed Description

A structure representing network configuration for the Sixdof sensor.

This structure contains the necessary information in order to connect to a Sixdof sensor.

The sensor_ip field is for cases where the user will add the entry to the ARP table themselves. Typically this is used on system where the basic network configuration will fail, for example in situations where the computer operating system is not in English.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.15 Sds::OpticalPeak Struct Reference

A structure representing a summary of the optical data for a single beacon.

`#include <SdsShutterControl.h>`

Public Attributes

- bool **has_id**

  peak has a parsed beacon id.
- bool **id_duplicate**

  this peak has a repeat id, usually indicating a reflection.
- uint32_t **id**

  the beacon id.
- double **height**

  the intensity of the peak [0, 255].
- double **background**

  a sample of the intensity of the background in the locality of the peak.
- char **carnea**

  the id of the carnea sensor ['C', 'L', 'P'].

4.15.1 Detailed Description

A structure representing a summary of the optical data for a single beacon.

This data represents a summary of the optical data corresponding to to a single beacon seen by a single sensor. This data can be used to customize shutter control.

The documentation for this struct was generated from the following file:

- SdsShutterControl.h

4.16 Sds::Falcon::Pose6Dof Struct Reference

A structure representing a 6DOF pose.

`#include <SdsFalcon.h>`

Public Attributes

- bool **valid** { false }

  flag indicating if the pose is valid. This can be False when calling the getPose6Dof function before a valid pose is obtained.
- double **x**

  x-coordinate in meters.
- double **y**

  y-coordinate in meters.
- double **z**

  z-coordinate in meters.
- double **qw**

  quaternion w component.
- double **qx**

  quaternion x component.
- double **qy**

  quaternion y component.
- double **qz**

  quaternion x component.
- float **var_x**
- float **var_y**
- float **var_z**
- float **var_h**
- float **var_p**
- float **var_r**

4.16.1 Detailed Description

A structure representing a 6DOF pose.

This structure contains 6DOF (6 degree of freedom) pose information, specifically the pose of the Sixdof sensor with respect to the map. The pose is represented by position (x, y, z) in meters, and orientation as a quaternion (qx, qw, qy, qz). Pose accuracy is represented as variance of the position (var_x, var_y, var_z) in meters squared, and the variance of the orientation (var_h, var_p, var_r) in radians squared.

This is the return type for the 6DOF tracking data.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.17 Sds::Falcon::RelativeAngle Struct Reference

A structure representing the relative angle of a beacon.

`#include <SdsFalcon.h>`

Public Attributes

- double **x_angle**
  angular offset in the x-axis in radians.
- double **z_angle**
  angular offset in the z-axis in radians.
- uint32_t **id**
  unique id of the beacon.
- double **intensity**
  light intensity of the beacon, this can be used to roughly indicate distance.
- double **width**
  width of the detected peak in pixels, also useful for indicating distance.

4.17.1 Detailed Description

A structure representing the relative angle of a beacon.

This structure contains the angle of a single beacon with respect to the sensor. Where x_angle is the angular offset along the x-axis, and z_angle is the angular offset along the z-axis.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.18 Sds::Falcon::StatusMessage Struct Reference

A structure representing a status message.

`#include <SdsFalcon.h>`

Public Attributes

- Severity **severity**
  severity level of the status message.
- CallingLayer **layer**
  calling layer for the status message.
- uint8_t **event_code**
  event code of the status message.
- std::string **message**
  verbose status message.

4.18.1 Detailed Description

A structure representing a status message.

This structure contains information about a status message, including its severity, calling layer, event code, and the content of the message.

The documentation for this struct was generated from the following file:

- SdsFalcon.h

4.19 Sds::Falcon::Version Class Reference

A class representing the version of either the shared library, or the sensor firmware.

`#include <SdsFalcon.h>`

Public Member Functions

- **Version** (uint8_t major, uint8_t minor, uint8_t patch)
- uint8_t getMajor () const

  Get the major component of the version.
- uint8_t getMinor () const

  Get the minor component of the version.
- uint8_t getPatch () const

  Get the patch component of the version.
- bool isEqual (uint8_t major, uint8_t minor, uint8_t patch) const

  Check if the version is equal to the specified components.
- bool isAtLeast (uint8_t major, uint8_t minor, uint8_t patch) const

  Check if the version is at least the specified version.
- std::string getString () const

  Get a string representation of the version.

4.19.1 Detailed Description

A class representing the version of either the shared library, or the sensor firmware.

This class provides utility functions around a system version (major, minor, patch), either the version of the shared library, or the sensor firmware version.

4.19.2 Member Function Documentation

4.19.2.1 getMajor()

`uint8_t Sds::Falcon::Version::getMajor ( ) const`

Get the major component of the version.

Returns

The major component of the version as a uint8_t.

4.19.2.2 getMinor()

```
uint8_t Sds::Falcon::Version::getMinor ( ) const
```

Get the minor component of the version.

Returns

The minor component of the version as a uint8_t.

4.19.2.3 getPatch()

```
uint8_t Sds::Falcon::Version::getPatch ( ) const
```

Get the patch component of the version.

Returns

The patch component of the version as a uint8_t.

4.19.2.4 getString()

```
std::string Sds::Falcon::Version::getString ( ) const
```

Get a string representation of the version.

Get a string representation of the version.

Returns

A string representing the version.

4.19.2.5 isAtLeast()

```
bool Sds::Falcon::Version::isAtLeast (
            uint8_t major,
            uint8_t minor,
            uint8_t patch ) const
```

Check if the version is at least the specified version.

Parameters

major	The major component to compare.
minor	The minor component to compare.
patch	The patch component to compare.

Returns

True if the version is at least the specified version, false otherwise.

4.19.2.6 isEqual()

```
bool Sds::Falcon::Version::isEqual (
            uint8_t major,
            uint8_t minor,
            uint8_t patch ) const
```

Check if the version is equal to the specified components.

Parameters

major	The major component to compare.
minor	The minor component to compare.
patch	The patch component to compare.

Returns

True if the version is equal to the specified components, false otherwise.

The documentation for this class was generated from the following file:

- SdsFalcon.h

Chapter 5

File Documentation

5.1 DevFeatures.h File Reference

Header file containing declarations for advanced Dev Features of the Sixdof Falcon library.

```
#include ¨SdsFalcon.h¨
```

Classes

- class Sds::Falcon::DevFeatures

  Class for managing runtime parameters of the Sixdof Falcon tracking system.

Macros

- #define DEV_FEATURE_LIST(F)
- #define **MAKE_PARAM_TYPE_ENUM**(name, default, min, max, description) name,

Enumerations

- enum class Sds::Falcon::ParamType : int { **Unknown** = 0 , **MAKE_PARAM_TYPE_ENUM** }

  An enum representing the parameters that can be set by DevFeatures.

Functions

- SDS_FALCON_API DevFeatures * Sds::Falcon::createDevFeatures ()

  Create an instance of DevFeatures class.
- SDS_FALCON_API bool Sds::Falcon::loadDevFeatures (DevFeatures *devFeatures, const std::string &path)

  Load an XML file with the Development Features.
- SDS_FALCON_API void **Sds::Falcon::saveDevFeatures** (DevFeatures *devFeatures, const std::string &path)

  Save an XML file with the Development Features.
- SDS_FALCON_API void **Sds::Falcon::loadDefaultDevFeatures** (DevFeatures *devFeatures)

  Load the Development Features with their default values.

5.1.1 Detailed Description

Header file containing declarations for advanced Dev Features of the Sixdof Falcon library.

This file contains the tools needed to customize the Sixdof Falcon tracking system parameters. Before changing parameters please contact the Sixdof team!

5.1.2 Macro Definition Documentation

5.1.2.1 DEV_FEATURE_LIST

```
#define DEV_FEATURE_LIST(
              F )
```

Value:

```
      F(std_xyz,                   0.005,    0.0,  1.0,  "EKF process noise x,y,z (m)") \
      F(std_xyz_f,                 0.5,      0.0,  1.0,  "EKF process noise x,y,z factor (%)") \
      F(damp_factor_ms,            0.46,     0.0,  1.0,  "Ekf velocity dampening factor (ms)") \
      F(sigma_gyro_offset,         0.001,    0.0,  1.0,  "Gyro noise (deg)") \
      F(sigma_gyro_factor,         0.005,    0.0,  1.0,  "Gyro noise factor (%)") \
      F(sigma_gyro_offset_unsolved, 0.06,    0.0,  1.0,  "Ekf gyro noise for when the gyro bias
hasn't been solved yet (deg)") \
      F(sigma_gyro_factor_unsolved, 0.05,    0.0,  1.0,  "Ekf gyro noise factor for when the gyro
bias hasn't been solved yet (%)") \
      F(use_damp_factor,           1.0,      0.0,  1.0,  "Ekf use dampening factor") \
      F(use_2d_extrapolation,      1.0,      0.0,  1.0,  "Ekf use 2d extrapolation for high elevation
peaks") \
      F(process_noise_idx,         0.3,      0.0,  1.0,  "Peak process noise (pixel)") \
      F(process_noise_v,           0.004625, 0.0,  1.0,  "Peak process noise velocity (pixel /
frame)") \
      F(meas_noise,                0.5,      0.0,  1.0,  "Peak measurement noise (pixel)") \
      F(localization_cooldown_sec, 0.02,     0.0,  1.0,  "Localization retry cooldown (sec)") \
      F(localization_features,     3.0,      3.0,  10.0, "P3P Localization minimum number of features
observed")\
```

5.1.3 Enumeration Type Documentation

5.1.3.1 ParamType

```
enum class Sds::Falcon::ParamType :  int  [strong]
```

An enum representing the parameters that can be set by DevFeatures.

An enum representing the parameters that can be set by DevFeatures.

5.1.4 Function Documentation

5.1.4.1 createDevFeatures()

```
SDS_FALCON_API DevFeatures * Sds::Falcon::createDevFeatures ( )
```

Create an instance of DevFeatures class.

Returns

A pointer to a DevFeatures class.

5.1.4.2 loadDevFeatures()

```
SDS_FALCON_API bool Sds::Falcon::loadDevFeatures (
            DevFeatures * devFeatures,
            const std::string & path )
```

Load an XML file with the Development Features.

Returns

False if the load was unsuccessful, true if successful

5.2 DevFeatures.h

Go to the documentation of this file.

```
00001
00010 #pragma once
00011 #include "SdsFalcon.h"
00012
00013 namespace Sds {
00014     namespace Falcon {
00015
00016         // name, default, min, max, description
00017         #define DEV_FEATURE_LIST(F) \
00018             F(std_xyz,                    0.005,    0.0,  1.0,  "EKF process noise x,y,z (m)") \
00019             F(std_xyz_f,                  0.5,      0.0,  1.0,  "EKF process noise x,y,z factor (%)")
    \
00020             F(damp_factor_ms,             0.46,     0.0,  1.0,  "Ekf velocity dampening factor (ms)")
    \
00021             F(sigma_gyro_offset,          0.001,    0.0,  1.0,  "Gyro noise (deg)") \
00022             F(sigma_gyro_factor,          0.005,    0.0,  1.0,  "Gyro noise factor (%)") \
00023             F(sigma_gyro_offset_unsolved, 0.06,     0.0,  1.0,  "Ekf gyro noise for when the gyro bias
    hasn't been solved yet (deg)") \
00024             F(sigma_gyro_factor_unsolved, 0.05,     0.0,  1.0,  "Ekf gyro noise factor for when the
    gyro bias hasn't been solved yet (%)") \
00025             F(use_damp_factor,            1.0,      0.0,  1.0,  "Ekf use dampening factor") \
00026             F(use_2d_extrapolation,       1.0,      0.0,  1.0,  "Ekf use 2d extrapolation for high
    elevation peaks") \
00027             F(process_noise_idx,          0.3,      0.0,  1.0,  "Peak process noise (pixel)") \
00028             F(process_noise_v,            0.004625, 0.0,  1.0,  "Peak process noise velocity (pixel /
    frame)") \
00029             F(meas_noise,                 0.5,      0.0,  1.0,  "Peak measurement noise (pixel)") \
00030             F(localization_cooldown_sec,  0.02,     0.0,  1.0,  "Localization retry cooldown (sec)") \
00031             F(localization_features,      3.0,      3.0,  10.0, "P3P Localization minimum number of
    features observed")\
00032
00039         enum class ParamType : int {
00040             Unknown = 0,
00041             #define MAKE_PARAM_TYPE_ENUM(name, default, min, max, description) name,
00042             DEV_FEATURE_LIST(MAKE_PARAM_TYPE_ENUM)
00043             #undef MAKE_PARAM_TYPE_ENUM
00044         };
00045
00046
00053         class SDS_FALCON_API DevFeatures {
00054         public:
00055             virtual ~DevFeatures() { }
00056
00060             virtual void setParam(ParamType, double) = 0;
00061
00065             virtual double getParam(ParamType) = 0;
00066         };
00067
00073         SDS_FALCON_API DevFeatures* createDevFeatures();
00074
00080         SDS_FALCON_API bool loadDevFeatures(DevFeatures* devFeatures, const std::string& path);
00081
00085         SDS_FALCON_API void saveDevFeatures(DevFeatures* devFeatures, const std::string& path);
00086
00090         SDS_FALCON_API void loadDefaultDevFeatures(DevFeatures* devFeatures);
00091     }
00092 }
```

5.3 SdsFalcon.h File Reference

Header file containing declarations for the Sixdof Falcon library.

```
#include <stdint.h>
#include <map>
#include <vector>
#include <string>
#include <memory>
#include <functional>
#include ¨SdsShutterControl.h¨
```

Classes

- class Sds::Falcon::Version

  A class representing the version of either the shared library, or the sensor firmware.
- struct Sds::Falcon::NetworkConfig

  A structure representing network configuration for the Sixdof sensor.
- struct Sds::Falcon::Beacon

  A structure representing a beacon with 3D coordinates and an id.
- struct Sds::Falcon::Pose6Dof

  A structure representing a 6DOF pose.
- struct Sds::Falcon::RelativeAngle

  A structure representing the relative angle of a beacon.
- struct Sds::Falcon::StatusMessage

  A structure representing a status message.
- struct Sds::Falcon::DynamicMessage

  A structure representing a dynamic message.
- struct Sds::Falcon::Heartbeat

  A structure representing health data of the SDK.
- struct Sds::Falcon::CurrentShutter

  A structure representing the current shutter value.
- struct Sds::Falcon::GyroOffset

  A structure representing a single gyro calibration offset for a specific sensor.
- struct Sds::Falcon::FieldOfViewReport

  A report of the beacons in the field of view.
- struct Sds::Falcon::DataPlaybackSettings

  The configuration settings of the data playback feature.
- struct Sds::Falcon::Config

  A structure representing configuration settings for the tracking.
- struct Sds::Falcon::ConfigGyroCalibration

  A structure representing configuration settings for gyro calibration.
- class Sds::Falcon::FalconManager

  Class for managing the Sixdof Falcon tracking system.

Macros

- #define **SDS_FALCON_API** __attribute__((visibility(¨default¨)))
- #define **DEPRECATED**(msg)

Typedefs

- typedef std::vector< RelativeAngle > Sds::Falcon::RelativeAngleCollection

  A structure representing a collection of RelativeAngle.
- typedef uint32_t Sds::Falcon::CallbackHandle

  A handle representing a registered callback function.

Enumerations

- enum class Sds::Falcon::Severity { Exception = 1 , Warning , Informative }

  An enum representing the severity level of a status message.
- enum class Sds::Falcon::CallingLayer { SdsFalcon = 1 , SdsCommLib , NetworkConnection , InternalAlgorithm }

  An enum representing the calling layer for a status message.
- enum class Sds::Falcon::FalconEventCode { TrackingNotInitalized , NetworkNotInitalized , GyroCalibrationRejected , CommunicationLoopNotClosed , DidNotReceiveFirmwareInfo , SensorFirmwareIsNotLatest , SensorFirmwareIsNotCompatible , DidNotReceive6DofMap , ConfigHasBothNetworkAndIp , PlaybackReachedEndOfFile }

  An enum representing event codes specific to the SdsFalcon calling layer.
- enum class Sds::Falcon::TrackingState { Initializing , Running , Error }

  An enum representing different states of the tracking.
- enum class Sds::Falcon::DataPlaybackMode { NONE = 1 , RECORD , PLAYBACK }

  An enum representing mode of the data playback feature.

Functions

- SDS_FALCON_API Version Sds::Falcon::getVersion ()

  Get the version of the SdsFalcon shared library.
- SDS_FALCON_API std::string Sds::Falcon::to_string (TrackingState state)

  Convert a TrackingState enum value to a string.
- SDS_FALCON_API void Sds::Falcon::saveConfig (const std::string &path, const Config &config)

  Save a configuration file.
- SDS_FALCON_API Config Sds::Falcon::loadConfig (const std::string &path)

  Load a configuration file.
- SDS_FALCON_API std::vector< Beacon > Sds::Falcon::loadMap (const std::string &path)

  Load a map file.
- SDS_FALCON_API std::unique_ptr< FalconManager > Sds::Falcon::createFalconManager ()

  Create an instance of FalconManager.

5.3.1 Detailed Description

Header file containing declarations for the Sixdof Falcon library.

This file contains the declarations for classes and data structures related to the Sixdof Falcon library. It provides functionality for managing the Sixdof sensor and the Sixdof Falcon tracking system.

5.3.2 Typedef Documentation

5.3.2.1 CallbackHandle

`Sds::Falcon::CallbackHandle`

A handle representing a registered callback function.

This typedef represents a callback handle used to identify registered callback functions. It is used to remove a specific callback.

5.3.2.2 RelativeAngleCollection

`Sds::Falcon::RelativeAngleCollection`

A structure representing a collection of RelativeAngle.

This structure contains multiple RelativeAngle estimates. All RelativeAngle estimates were calculated at the same and therefore bundled together.

This is the return type for RelativeAngle tracking data.

5.3.3 Enumeration Type Documentation

5.3.3.1 CallingLayer

`enum class Sds::Falcon::CallingLayer [strong]`

An enum representing the calling layer for a status message.

This enum defines the calling layers for status messages. Each layer indicates the source of a particular message.

Enumerator

SdsFalcon	Message from the top layer of the library. Generally indicates the library was used incorrectly.
SdsCommLib	Message from the UDP communication library, this is used to communicate with the sensor.
NetworkConnection	Message from the network connection stage.
InternalAlgorithm	Message from the internal Falcon algorithm.

5.3.3.2 DataPlaybackMode

`enum class Sds::Falcon::DataPlaybackMode [strong]`

An enum representing mode of the data playback feature.

This enumeration defines the operational mode of the data playback feature.

Enumerator

NONE	Real-time mode. Playback and recording are disabled.
RECORD	Real-time mode with recording enabled.
PLAYBACK	Playback mode. Recorded data is replayed instead of live sensor data. Playback begins automatically during initialize(); start() is not required.

5.3.3.3 FalconEventCode

```
enum class Sds::Falcon::FalconEventCode  [strong]
```

An enum representing event codes specific to the SdsFalcon calling layer.

This enum defines event codes that are specific to the SdsFalcon calling layer.

Enumerator

TrackingNotInitalized	indicates tracking was started before initialization.
NetworkNotInitalized	indicates that the initializeNetwork function was not called.
GyroCalibrationRejected	indicates gyro calibration values were rejected.
CommunicationLoopNotClosed	communication loop is not closed, check your firewall.
DidNotReceiveFirmwareInfo	did not receive firmware information from the sensor.
SensorFirmwareIsNotLatest	sensor firmware is not latest version, it is not guaranteed that all SDK features will work fully.
SensorFirmwareIsNotCompatible	sensor firmware is not compatible with current SDK, please contact Sixdof for an upgrade.
DidNotReceive6DofMap	did not receive beacon map information in the Config.
ConfigHasBothNetworkAndIp	received both the sensor network name and the sensor ip in the Config.
PlaybackReachedEndOfFile	data playback feature reached the end of file.

5.3.3.4 Severity

```
enum class Sds::Falcon::Severity  [strong]
```

An enum representing the severity level of a status message.

This enumeration defines severity levels for status messages.

Enumerator

Exception	Exception indicates that there was a error in operation.
Warning	Warning indicates that something went wrong but it is not critical.
Informative	Informative indicates general debugging information, these can be ignored.

5.3.3.5 TrackingState

```
enum class Sds::Falcon::TrackingState  [strong]
```

An enum representing different states of the tracking.

Enumerator

Initializing	tracking is currently initializing.
Running	tracking is currently running.
Error	tracking is currently in the error state.

5.3.4 Function Documentation

5.3.4.1 createFalconManager()

```
SDS_FALCON_API std::unique_ptr< FalconManager > Sds::Falcon::createFalconManager ( )
```

Create an instance of FalconManager.

This function creates a new instance of the FalconManager. When the FalconManager goes out of scope it will stop the sensor and be cleaned up automatically.

Returns

A unique pointer to a new instance of FalconManager.

5.3.4.2 getVersion()

```
SDS_FALCON_API Version Sds::Falcon::getVersion ( )
```

Get the version of the SdsFalcon shared library.

Returns

The Version object representing the library version.

5.3.4.3 loadConfig()

```
SDS_FALCON_API Config Sds::Falcon::loadConfig (
            const std::string & path )
```

Load a configuration file.

Load an XML file with the configuration settings. This enables the user to load a file containing the configuration instead of entering it manually. It can only be called before initialize is called. An exception is thrown if there is an error in loading the file.

Parameters

path	The path to the file containing the configuration.

Returns

Config

5.3.4.4 loadMap()

```
SDS_FALCON_API std::vector< Beacon > Sds::Falcon::loadMap (
            const std::string & path )
```

Load a map file.

Load an XML file with the map settings for the configuration of 6DOF tracking. This enables the user to load a file containing the map configuration instead of entering it manually. An exception is thrown if there is an error in loading the file. Note: This function only loads and returns the map data; it does not assign the result to the Config::map member.

Parameters

path	The path to the file containing the map configuration.

Returns

The map as a vector of Beacon objects.

5.3.4.5 saveConfig()

```
SDS_FALCON_API void Sds::Falcon::saveConfig (
            const std::string & path,
            const Config & config )
```

Save a configuration file.

Save an XML file with the configuration settings. This enables the user to save a file containing the configuration. An exception is thrown if there is an error in saving the file.

Parameters

path	The desired path of the file containing the configuration.
config	The configuration.

5.3.4.6 to_string()

```
SDS_FALCON_API std::string Sds::Falcon::to_string (
            TrackingState state )
```

Convert a TrackingState enum value to a string.

Parameters

state	TrackingState enum value.

Returns

string representation of the TrackingState.

5.4 SdsFalcon.h

Go to the documentation of this file.

```
00001 //Copyright © 2017-2026 Six Degrees Space Ltd.  All rights reserved.
00002 //Proprietary and Confidential.  Unauthorized use, disclosure or reproduction is strictly prohibited.
00003
00012 #pragma once
00013 #include <stdint.h>
00014 #include <map>
00015 #include <vector>
00016 #include <string>
00017 #include <memory>
00018 #include <functional>
00019
00020 #include "SdsShutterControl.h"
00021
00022 #ifdef _WIN32
00023     // Windows platform
00024     #ifdef SDS_EXPORTS
00025         #define SDS_FALCON_API __declspec(dllexport)
00026     #else
00027         #define SDS_FALCON_API __declspec(dllimport)
00028     #endif
00029 #else
00030     // Non-Windows platform
00031     #define SDS_FALCON_API __attribute__((visibility("default")))
00032 #endif
00033
00034 #if defined(__GNUC__) || defined(__clang__)
00035 #define DEPRECATED(msg) __attribute__((deprecated(msg)))
00036 #elif defined(_MSC_VER)
00037 #define DEPRECATED(msg) __declspec(deprecated(msg))
00038 #else
00039 #define DEPRECATED(msg)
00040 #endif
00041
00179 namespace Sds {
00180     namespace Falcon {
00181
00188         class SDS_FALCON_API Version {
00189         public:
00190             Version(uint8_t major, uint8_t minor, uint8_t patch);
00191
00196             uint8_t getMajor() const;
00197
00202             uint8_t getMinor() const;
00203
00208             uint8_t getPatch() const;
00209
00218             bool isEqual(uint8_t major, uint8_t minor, uint8_t patch) const;
00219
00228             bool isAtLeast(uint8_t major, uint8_t minor, uint8_t patch) const;
00229
00236             std::string getString() const;
00237
00238         private:
00239             uint8_t _major, _minor, _patch;
00240         };
00241
00246         SDS_FALCON_API Version getVersion();
00247
00258         struct SDS_FALCON_API NetworkConfig {
00259             uint16_t sensor_id;
00260             std::string sensor_network_name { "" };
00261             std::string sensor_ip { "" };
00262             uint16_t udp_read_port { 0 };
00263         };
00264
```

```
00271         struct SDS_FALCON_API Beacon {
00272             double x;
00273             double y;
00274             double z;
00275             uint32_t id;
00276         };
00277
00288         struct SDS_FALCON_API Pose6Dof {
00289             bool valid { false };
00290             double x;
00291             double y;
00292             double z;
00293             double qw;
00294             double qx;
00295             double qy;
00296             double qz;
00297             float var_x, var_y, var_z; // m^2
00298             float var_h, var_p, var_r; // rad^2
00299         };
00300
00308         struct SDS_FALCON_API RelativeAngle {
00309             double x_angle;
00310             double z_angle;
00311             uint32_t id;
00312             double intensity;
00313             double width;
00314         };
00315
00325         typedef std::vector<RelativeAngle> RelativeAngleCollection;
00326
00333         enum class Severity {
00334             Exception = 1,
00335             Warning,
00336             Informative,
00337         };
00338
00346         enum class CallingLayer {
00347             SdsFalcon = 1,
00348             SdsCommLib,
00349             NetworkConnection,
00350             InternalAlgorithm,
00351         };
00352
00359         enum class FalconEventCode {
00360             TrackingNotInitalized,
00361             NetworkNotInitalized,
00362             GyroCalibrationRejected,
00363             CommunicationLoopNotClosed,
00364             DidNotReceiveFirmwareInfo,
00365             SensorFirmwareIsNotLatest,
00366             SensorFirmwareIsNotCompatible,
00367             DidNotReceive6DofMap,
00368             ConfigHasBothNetworkAndIp,
00369             PlaybackReachedEndOfFile,
00370         };
00371
00379         struct SDS_FALCON_API StatusMessage {
00380             Severity severity;
00381             CallingLayer layer;
00382             uint8_t event_code;
00383             std::string message;
00384         };
00385
00393         struct SDS_FALCON_API DynamicMessage {
00394             bool valid { false };
00395             uint32_t beacon_id;
00396             std::string message;
00397         };
00398
00403         enum class TrackingState {
00404             Initializing,
00405             Running,
00406             Error,
00407         };
00408
00415         SDS_FALCON_API std::string to_string(TrackingState state);
00416
00424         struct SDS_FALCON_API Heartbeat {
00425             bool sensor_communication_ok { false };
00426             TrackingState current_tracking_state { TrackingState::Initializing };
00427         };
00428
00436         struct SDS_FALCON_API CurrentShutter {
00437             uint8_t shutter_value;
00438         };
00439
00446         struct SDS_FALCON_API GyroOffset {
```

```
00447             uint16_t sensorId { 0 };
00448             std::string datetime { "" };
00449             int8_t temperature { 0 };
00450             int16_t gyro_x { 0 };
00451             int16_t gyro_y { 0 };
00452             int16_t gyro_z { 0 };
00453             uint16_t standard_dev { 0 };
00454         };
00455
00462         struct SDS_FALCON_API FieldOfViewReport {
00463             std::map<uint32_t, int> seenBeacons;
00464         };
00465
00472         enum class DataPlaybackMode {
00473             NONE = 1,
00474             RECORD,
00475             PLAYBACK,
00476         };
00477
00486         struct SDS_FALCON_API DataPlaybackSettings {
00487             DataPlaybackMode mode { DataPlaybackMode::NONE };
00488             std::string path { "" };
00489         };
00490
00499         struct SDS_FALCON_API Config {
00500             NetworkConfig networkConfig;
00501             std::vector<Beacon> map;
00502             std::vector<GyroOffset> gyroCalibration;
00503             bool skip_gyro_calibration { true };
00504             bool no_gyro_mode { false };
00505         };
00506
00517         SDS_FALCON_API void saveConfig(const std::string& path, const Config& config);
00518
00530         SDS_FALCON_API Config loadConfig(const std::string& path);
00531
00543         SDS_FALCON_API std::vector<Beacon> loadMap(const std::string& path);
00544
00552         struct SDS_FALCON_API ConfigGyroCalibration {
00553             int duration_seconds { 30 };
00554         };
00555
00563         typedef uint32_t CallbackHandle;
00564
00584         class SDS_FALCON_API FalconManager {
00585         public:
00586             virtual ~FalconManager() { }
00587
00597             virtual bool initialize(const Config&) = 0;
00598
00609             virtual bool initialize(const Config&, const DataPlaybackSettings&) = 0;
00610
00614             virtual void start() = 0;
00615
00619             virtual void stop() = 0;
00620
00624             virtual Version getFirmwareVersion() = 0;
00625
00652             virtual std::vector<GyroOffset> getGyroCalibration(const ConfigGyroCalibration&) = 0;
00653
00654             virtual Pose6Dof getPose6Dof() = 0;
00655
00656             virtual RelativeAngleCollection getRelativeAngleCollection() = 0;
00657
00658             virtual StatusMessage getStatusMessage() = 0;
00659
00660             virtual DynamicMessage getDynamicMessage() = 0;
00661
00662             virtual FieldOfViewReport getFieldOfViewReport() = 0;
00663
00664             virtual CurrentShutter getCurrentShutter() = 0;
00665
00666             virtual CallbackHandle registerPose6DofCallback(const std::function<void(Pose6Dof)>&) = 0;
00667
00668             virtual CallbackHandle registerRelativeAngleCallback(const
      std::function<void(RelativeAngleCollection)>&) = 0;
00669
00670             virtual CallbackHandle registerMessageCallback(const std::function<void(StatusMessage)>&)
      = 0;
00671
00672             virtual CallbackHandle registerDynamicMessageCallback(const
      std::function<void(DynamicMessage)>&) = 0;
00673
00674             virtual CallbackHandle registerFieldOfViewReportCallback(const
      std::function<void(FieldOfViewReport)>&) = 0;
00675
00676             virtual CallbackHandle registerHeartbeatCallback(const std::function<void(Heartbeat)>&) =
```

```
      0;
00677
00678             virtual CallbackHandle registerCurrentShutterCallback(const
      std::function<void(CurrentShutter)>&) = 0;
00679
00680             virtual void removePose6DofCallback(const CallbackHandle&) = 0;
00681
00682             virtual void removeRelativeAngleCallback(const CallbackHandle&) = 0;
00683
00684             virtual void removeMessageCallback(const CallbackHandle&) = 0;
00685
00686             virtual void removeDynamicMessageCallback(const CallbackHandle&) = 0;
00687
00688             virtual void removeFieldOfViewCallback(const CallbackHandle&) = 0;
00689
00690             virtual void removeHeartbeatCallback(const CallbackHandle&) = 0;
00691
00692             virtual void removeCurrentShutterCallback(const CallbackHandle&) = 0;
00693
00702             virtual void swapMap(const std::vector<Beacon>& map) = 0;
00703
00710             virtual void setShutterControl(std::shared_ptr<IShutterControl> shutterControl) = 0;
00711
00722             void loadShutterPlugin(const std::string& path);
00723
00730             void loadDefaultShutterControl();
00731         };
00732
00741         SDS_FALCON_API std::unique_ptr<FalconManager> createFalconManager();
00742     }
00743 }
```

5.5 SdsShutterControl.h File Reference

Header file containing declarations relating to the Sixdof sensors shutter control.

```
#include <stdint.h>
#include <vector>
#include <memory>
```

Classes

- class Sds::ISetShutterListener

  Listener interface that allows IShutterControl to pass forward requests to change the shutter.
- struct Sds::OpticalPeak

  A structure representing a summary of the optical data for a single beacon.
- class Sds::IShutterControl

  Shutter control base class.

Macros

- #define **SDS_API** __attribute__((visibility("default")))

5.5.1 Detailed Description

Header file containing declarations relating to the Sixdof sensors shutter control.

This file contains the declarations for classes and data structures related to controlling the shutter of the Sixdof sensor. This header file is shared between the Sixdof Falcon, and Sixdof Cheetah software products.

The functionality provided in this file is optional, the default shutter behaviour will work for most use cases. Users with more complicated deployment scenarios, such as cases where; there are LEDs of different brightnesses, with very fast moving targets, or situations with complicated lighting features, may use the functionality presented here to develop custom shutter control logic.

The Sixdof sensor has 17 shutter values, 0 to 16, where 0 is the minimum shutter exposer time, and 16 is the maximum shutter exposure time. Additionally, there is a mode called ¨Auto-shutter¨ which allows the sensor itself to optimize the shutter.\ Auto-shutter mode is acceptable in many use cases, however it does not use any information other than the raw data to set the shutter value. This limitation makes Auto-shutter a poor choice in environments where there may be lots of background interference, due to sunlight.

The Sixdof Falcon, and Sixdof Cheetah, solutions both provide default shutter behaviour that take into consideration more information, such as the beacon ids that are currently being used. Therefore, the default shutter behaviour preforms better than Auto-shutter in many circumstances.

The following classes enable the user to override the default shutter behaviour for advanced deployment use cases.

In order to customize the shutter logic, inherit from the IShutterControl base class. Then the setShutterAuto, and setShutterFixed functions can be used to change the shutter. Note, it does take a small period of time for the sensor to change its shutter, so these functions should not be called more than every 20ms.

Both SdsFalcon and SdsCheetah can accept a std::shared_ptr<IShutterControl>. Additionally, the custom shutter logic can be compiled into a plugin that can be compiled and provided to SdsFalcon or SdsCheetah. The advantage of a compiled plugin is that it can be loaded into the Sixdof Falcon Studio, or Sixdof Cheetah Studio GUI applications for testing. To make a plugin implement a shared library with the following fuction: extern ¨C¨ IShutterControl* createShutterControl()

5.6 SdsShutterControl.h

Go to the documentation of this file.

```
00001
00032 #pragma once
00033 #include <stdint.h>
00034 #include <vector>
00035 #include <memory>
00036
00037 #ifdef _WIN32
00038     // Windows platform
00039     #ifdef SDS_EXPORTS
00040         #define SDS_API __declspec(dllexport)
00041     #else
00042         #define SDS_API __declspec(dllimport)
00043     #endif
00044 #else
00045     // Non-Windows platform
00046     #define SDS_API __attribute__((visibility("default")))
00047 #endif
00048
00049 namespace Sds {
00050
00058     class SDS_API ISetShutterListener {
00059     public:
00060         virtual void setShutterAuto(uint8_t min_shutter, uint8_t max_shutter) = 0;
00061
00062         virtual void setShutterFixed(uint8_t shutter) = 0;
00063     };
00064
00072     struct OpticalPeak {
```

```
00073         bool has_id;
00074         bool id_duplicate;
00075         uint32_t id;
00076         double height;
00077         double background;
00078         char carnea;
00079     };
00080 
00087     class SDS_API IShutterControl {
00088     public:
00089 
00090         virtual ~IShutterControl() = default;
00091 
00100         virtual void updateShutter(uint8_t currentShutter, const std::vector<OpticalPeak>& peaks) = 0;
00101 
00107         void setShutterAuto(uint8_t min_shutter, uint8_t max_shutter) {
00108             if (listener) { listener->setShutterAuto(min_shutter, max_shutter); }
00109         }
00110 
00117         void setShutterFixed(uint8_t shutter) {
00118             if (listener) { listener->setShutterFixed(shutter); }
00119         }
00120 
00126         void registerSetShutterListener(ISetShutterListener* l) {
00127             listener = l;
00128         };
00129     private:
00130         ISetShutterListener* listener;
00131     };
00132 
00133 }
```